

Proof Search as Concurrent Reaction

The Rho Calculus and the Category of Continued Interactive GSLTs
as a Substrate for Proof Objects with Internalized, Cost-Accounted Search

L. G. Meredith
F1R3FLY.io

May 31, 2026

Abstract

Proof objects in contemporary proof assistants are variants of the typed λ -calculus, so their dynamics — normalization, equivalently cut elimination — is sequential and confluent. Concurrency, when these systems are scaled out, is supplied by an orchestration layer built *on top* in a different model of computation, leaving proof search external to proof dynamics. We argue that the category of continued interactive graph-structured lambda theories (**ciGSLT**) is a class of computational models better suited to hosting proof objects, in which proof search is internalized as concurrent proof dynamics, and that the rho calculus is the distinguished object of this category on which every requirement is native. The base case of the proposal already exists as a theorem — the Caires–Pfenning–Wadler correspondence between linear-logic proofs and session-typed processes, under which cut elimination is communication — and our contribution is two generalizations stacked on that base: replacing the fixed logic with an arbitrary **ciGSLT** whose propositions are modal (Hennessy–Milner) formulas over its labelled transition system, and pushing from proof *normalization* to proof *search*. The pivot is a single identification: an unproven obligation is a pending communication. Reflection makes obligations first-class, collapsing the object/meta distinction and placing the orchestrator inside the dynamics. The cost-accounting endofunctor of the prior notes then supplies search control in two registers — an economic one in which inference rules are priced and budgets replace coordinator algorithms, and a biological one in which mortality (deadlock-by-starvation) prunes unproductive branches without a coordinator and metabolism reallocates proof-effort under selection. We close with three theorem-shaped targets: encoding-adequacy of decentralized search, fairness-and-budget completeness in the limit, and proof identity as bisimulation.

1 The problem: orchestration outside the dynamics

In Lean, Coq, Agda and their relatives, a proof object is a term of a typed λ -calculus — the Calculus of Inductive Constructions, or for some a core in the spirit of System F — and the Curry–Howard correspondence identifies proof normalization with β -reduction and, equivalently, cut elimination with reduction in the term calculus. This is the deepest and most useful fact in the area, and it has a structural consequence that is rarely named as a limitation because it is usually a virtue: proof *dynamics is sequential*. A proof term is a finished plan whose reduction is canonical up to the Church–Rosser property; there is no room in the reduction relation itself for genuine choice, and there are no open ends in a closed term that *interact*.

When such systems are scaled out — parallel elaboration, distributed proof obligations, hammer-style search across many backends, learned premise selection feeding many provers — concurrency is introduced by building an orchestration system *on top*, in a different model of computation: a scheduler, a worklist, a message layer, a tactic interpreter that constructs

and rewrites terms from the outside. The orchestration lives beside the proof dynamics, not within it. This is not an accident of engineering. It is forced by the two properties just named, and the first contribution of this note is to say precisely why.

We then argue the positive thesis. The category of continued interactive graph-structured lambda theories, developed in the prior notes of this series as the home of the cost-accounting endofunctor and its internalization adjunction, is a class of models in which proof objects can be represented so that proof search is *internalized* as concurrent proof dynamics. Interaction among proof objects yields a decentralized and distributed proof search with no overarching coordinator. Because proof search is undecidable in general, hosting proof representation and proof search in a category where cost accounting is available — and, on Turing-complete bases, *internalizable* — supports both economically controlled search tactics and biologically and evolutionarily inspired ones. The rho calculus is the distinguished object on which all of this is native rather than aspirational.

2 The base case is a theorem

The proposal is not free-floating; its restriction to one fixed logic is an established correspondence. Caires and Pfenning showed that a session type discipline for the π -calculus corresponds exactly to the sequent calculus of dual intuitionistic linear logic, with process reduction simulating proof reduction and conversely [1]. Wadler recast the classical case as CP, a calculus in which the propositions of classical linear logic *are* session types, communication *is* cut elimination, and deadlock freedom follows from the logical correspondence [2]. Both descend from the older reading of linear logic proofs as processes [3]. So “proof dynamics as concurrent interaction” is, at the level of normalization, an isomorphism rather than a hope.

Two features of this base case are exactly what our generalizations must lift. First, it is pinned to a fixed logic: the propositions are linear-logic formulas, and the calculus is a session-typed fragment of the π -calculus [4]. Second, it concerns the dynamics of an *already-found* proof: cut elimination normalizes a proof that exists, it does not search for one. The genuinely new step is the second, and most of the technical weight of the programme falls there.

3 Why the λ -calculus forces the orchestrator outside

It is worth stating the negative result crisply, since it is the entire motivation. Two properties of the core calculus force proof search into a separate metalanguage.

Confluence. Reduction of a CIC term is canonical up to Church–Rosser, so the dynamics has no room for genuine choice. But proof search *is* choice: which rule to apply, which hypothesis to use, which term to instantiate. A confluent reduction relation cannot express the branching that search explores, so the branching must be managed elsewhere.

Closedness. A λ -term has no open ends that interact. A partial proof can therefore be represented only as a term with a hole, and a hole can be filled solely by an *external* agent that inspects the term and plugs it. There is no notion, internal to reduction, of an incomplete proof reaching out for what it lacks.

Both missing affordances — nondeterministic choice and interactive open-endedness — are precisely what a process calculus has and a λ -calculus does not. Hence the meta/object split (Lean’s elaboration engine and metaprogramming monads, Coq’s tactic language) is not a design taste but the only place choice and open-endedness can live when proof objects are confluent closed terms.

4 Obligation is communication

The pivot of the whole proposal is one identification.

An unproven obligation is a pending communication.

In the session-typed base case this is already literal: a free channel typed by a formula A is an open hypothesis or goal of type A ; the typing context lists the obligations as channel types. Generalize the carrier from the session-typed π -calculus to an arbitrary continued interactive GSLT and the open goals of a partial proof become exactly the free names and pending inputs of a proof-process. In the implementation vocabulary of the GSLT-to-rho compiler of the prior notes, an open goal is a *Hole*-bud and a rule scheme awaiting instantiation is a *Schema*-bud.

This identification dissolves the level distinction. Proof checking and proof search are not two activities at two levels; they are *one* dynamics observed at different stages of saturation. Search is the dynamics of partial, open proof-processes seeking interaction partners — lemmas, axioms, rule instances. Checking is the same dynamics after no obligations remain pending. A completed proof is a configuration with no free obligations and no enabled interaction: a normal form in the behavioral sense.

Reflection is what makes the identification *usable* rather than merely true. In the rho calculus, quotation $@P$ turns a process into a name and dereference $*x$ turns a name back into the process it quotes [5]. A proof-process can therefore reify its own open obligations as data, route them, inspect them, transform them, and run them. This is what places the orchestrator *inside* the dynamics: the object language of proofs and the metalanguage of search-strategy live in one calculus, because “manipulate an obligation” and “manipulate a process as data” are the same operation. The session-typed base case did not need reflection because normalization does not need it. Search does.

5 Proof search as concurrent reaction

The operational picture is the chemical-abstract-machine reading of rewriting [9], which is a hair’s breadth from a GSLT presented as types, equations, and rewrites [10]. Goals, rules, lemmas and axioms float as molecules in a solution and react when a redex is enabled. The disjunctive structure of search — the OR-node, the choice of which rule to try — is competing communication on a shared channel, the nondeterministic choice the λ -calculus could not express. The conjunctive structure — the AND-node, the several premises a rule demands — is parallel composition, and in the rho calculus specifically it is a *join pattern*:

$$\text{for } (y_1 \leftarrow a_1 \ \& \ \cdots \ \& \ y_n \leftarrow a_n) P$$

fires only when all n input obligations are simultaneously available. The multi-premise rule is the join; the join firing is the coalition of subproofs all succeeding together. This is the Curry–Howard reading between Coalition Logic proofs and rho joins from the consensus-synthesis note, surfacing here as the native realization of an AND-node: no coordinator assembles the conjunction, the join does.

The reaction itself is the structural-congruence-plus-communication semantics of the calculus: parallel composition with unit 0 is the commutative monoid that *is* the solution, the communication rule is the reaction, and reflection is what lets the molecules carry one another’s recipes. The decentralized soup is not a metaphor laid over the model; it is the operational semantics read aloud, with no agenda, no global worklist, and locally enabled reactivity.

This regime has honest precedents, and the note should name them rather than claim virgin ground. Concurrent constraint programming, with its ask/tell on a shared store, is already a decentralized, coordinator-free proof search and a foundational instance of “proof search as

the soup dynamics” [8]. The uniform-proofs and abstract-logic-programming tradition makes proof search the operational semantics of a logic, with the connectives read as fixed search instructions [7]. The deltas of the present proposal over both are exactly: reflection, which collapses object and meta; and an internalized cost layer, which controls and selects.

6 Rho versus CP: the inversion

The reason the rho calculus beats Wadler’s CP for *this* task is close to a clean inversion, and it is worth stating as such, because CP’s strengths are precisely the wrong strengths here.

CP is a triumph of correspondence at the level of normalization, and it pays for that tightness in exactly the currency that ruins it as a search substrate. Well-typed CP is deadlock-free by theorem; its cut-reduction fragment is confluent and strongly normalizing; and what looks like choice is typed into race-free internal and external choice [2, 1]. Those are the properties that make cut elimination behave like a finished computation. But proof *search* requires the three things a type discipline of this kind is built to prohibit:

1. *Nonconfluent choice* — real races between candidate rules, since the order and outcome of choices is the content of search.
2. *Deadlock* — because deadlock-by-starvation is the pruning mechanism; mortality is not a failure mode here, it is the garbage collector.
3. *Potential divergence* — because the set of proofs is recursively enumerable but not recursive, so any sound and complete search must be permitted not to terminate.

CP disciplines all three away statically, by typing, as prohibitions. The rho calculus keeps all three live and disciplines them dynamically, by an economy. For a search substrate this is the correct control surface: the pathologies on which search depends are exactly the ones a type discipline exists to forbid. The choice is not of a worse-behaved calculus but of the one whose control is in the right place.

Reflection is the second decisive gap. CP and the intuitionistic session calculus are first-order in the relevant sense: channels carry channels, not proof terms one can run as data. To obtain proofs-as-data one bolts on a higher-order layer or a metalanguage, reintroducing the object/meta split the programme exists to dissolve. In the rho calculus, $@(\cdot)$ and $*(\cdot)$ are that layer, inside one calculus.

There is a third point, and it converts a liability into an asset. Moving proof dynamics into a nonconfluent model costs the canonicity of normal forms, so proof identity can no longer be $\beta\eta$ -equality; the right notion becomes bisimulation. In CP this is a genuine imposition, since the typed terms *want* an observational identity one would be overriding. In the rho calculus, weak and barbed bisimulation is already the home equivalence; there is no canonical-normal-form identity to surrender. So “proof identity is bisimulation” stops being a redefinition and becomes the default reading, and it lands directly on the modal logic: two proofs are the same exactly when no Hennessy–Milner formula separates their behavior [6], which is the equivalence the algorithmic-scientists thread already singles out as the finest learnable one. The cost incurred on a generic **ciGSLT** is free on rho specifically.

7 The cost layer: economy and biology

The cost-accounting endofunctor **Cost** of the prior notes adjoins a signature monoid, a token stack, a signed-term wrapper, and replaces each rewrite by its token-gated family, so that no step fires without consuming matching fuel; on Turing-complete bases the internalization map $\mathbf{Cost}(G) \rightarrow G$ exhibits the metered calculus as a behavioral retract of the bare one, up to weak bisimulation. Applied to proof search this layer supplies control in two registers.

Economic control. Price each rule or tactic in phlogiston: a rewrite is cheap, a decision-procedure call is dear. A goal-process carries a budget and can fire only rules it can afford. Iterative deepening, best-first, and breadth-versus-depth become *budget policies* rather than coordinator algorithms. Because cost is internalized, a goal-process can reason about its own budget and reallocate — self-budgeting search, a meta-level strategy with no meta-level interpreter, since the meter lives in the same calculus as the proof.

Evolutionary control. The mortal-computation note identified *death as deadlock-by-starvation*: a process whose token stack is exhausted has no enabled rewrite and blocks permanently, and this configuration was native to the un-metered model all along. Read as proof search, an unproductive branch *prunes itself* — no coordinator decides to kill it; it fails to pay and deadlocks. That is the decentralization claim made mechanical. *Metabolism* supplies the rest: a proven lemma is a high-energy metabolite, a stable intermediate that downstream goal-processes consume; the splitters and joiners that reconcile token granularity are catabolism and anabolism, reallocating proof-fuel toward branches producing reusable structure; the gated rewrite is the thermodynamic coupling forbidding work without payment. With reflection, a tactic-process that closes goals cheaply can replicate into fresh contexts, with the phlogiston economy as the selection pressure — fitness internal, not an externally imposed objective. The variation operator is then the language-defining capability of the surrounding framework: mutating and recombining the rewrite rules of a defined proof-language mutates the tactics, with the language’s types, equations, and rewrites playing the role of a genome and the cost economy doing the selecting. This is Leo Buss’s account of why selection favours mortal organisms [11], transposed to proof search with a precise substrate rather than an analogy.

One economy. On the rho calculus, the blockchain gas economy, the proof-search budget, and the metabolic energy are not three economies. They are one phlogiston economy on one substrate. The node’s gas *is* a search budget *is* a metabolism. This is the payoff of doing the work on rho rather than on a bespoke calculus: the control layer already exists, is deployed, and is the same coin under all three readings.

8 Soundness, proof identity, and completeness in the limit

Soundness. The search moves are the rewrites; the rewrites are the bisimulation-preserving morphisms on which **Cost** was defined; preservation of bisimulation is preservation of the generated Hennessy–Milner property [6]. Hence each search step is sound with respect to the behavioral proposition it claims to preserve. The shift to accept is that proofs here prove *behavioral and modal* properties, not static type inhabitation. This is the natural home for verifying concurrent and distributed systems and a slightly awkward fit for, say, ordinary arithmetic; we lean into the former.

Proof identity. As noted, identity of proofs is bisimulation, which on rho is the home equivalence rather than an imposition. Two proof-processes denote the same proof when no modal formula separates them. This is coarser than $\beta\eta$ -equality and behavioral by design.

Completeness in the limit. The cleanest statement uses the internalization. Under fair scheduling and unbounded budget, the reachable closed configurations of the bare calculus are exactly the recursively enumerable set of proofs of the underlying theory — semi-decidability realized as reachability, the most a sound search can promise. Adding cost monotonically *restricts* reachability: every starvation-deadlock removes a branch, so completeness degrades gracefully rather than catastrophically, and sending the token supply to infinity recovers full

reachability. The two knobs are orthogonal and both must be maximized for completeness — fairness rules out *scheduling* starvation, budget rules out *resource* starvation — and the cost layer is exactly the dial between “complete and possibly divergent” and “bounded and self-pruning.”

Remark 1. Cost accounting never converts an undecidable search into a decidable one. What it converts is the *character* of nontermination: from “search may silently diverge” to “search consumes a controllable budget at a controllable rate.” This is the anytime, resource-bounded reframing one actually wants in practice, and it is the right place to host the undecidability of search.

9 Three theorem-shaped targets

Target 1 (Encoding and adequacy of decentralized search). A proof-search strategy specified as a global traversal of an AND/OR space is realized by a parallel composition of goal-processes and rule-processes in a continued interactive GSLT, such that the reachable terminal configurations correspond exactly to the proofs the strategy finds, and no process holds global search state. The AND-nodes are join patterns, the OR-nodes are competing communications, and the correspondence is up to bisimulation.

Target 2 (Fairness-and-budget completeness). Under a fairness condition on the labelled transition system and an unbounded token budget, the concurrent search is complete with respect to the underlying theory: every provable goal has a reachable closed configuration. Equivalently, the reachable closed configurations coincide with the recursively enumerable set of proofs. Finite budgets restrict this set monotonically, with each starvation-deadlock excising one branch.

Target 3 (Proof identity is bisimulation). Two proof-processes denote the same proof if and only if they are weakly bisimilar, equivalently if and only if no generated Hennessy–Milner formula separates them. The characterization factors through the modal logic so that “same behavior” and “same proof” provably coincide.

10 The wider category and the agenda

The reason “rho sits inside **ciGSLT**” earns its keep is that the cost endofunctor, the internalization adjunction, and the framework’s ability to *define* a proof-language-specific GSLT and compile it down to rho all live in that neighborhood. One can grow a logic-tailored GSLT, compile it into rho, and run its proof search in the same soup as everything else; one can let the language-defining layer evolve the rewrite rules, which is to say evolve the tactics, with phlogiston as the selection pressure. The distinguished object gives nativity — concurrency and joins for AND/OR, reflection for the meta-collapse, deadlock for mortality-as-pruning, bisimulation as the free proof-identity, one economy for gas, budget, and metabolism — and the surrounding category gives room to explore: other objects, other logics, other cost structures, with the bisimulation-preserving morphisms as the connecting tissue.

The note that follows should open exactly here, on rho versus CP framed as the inversion of §6, establish the obligation-is-communication identification of §4 as its hinge, and only then generalize back up to the category, carrying the three targets of §10 as the formal obligations the programme takes on.

References

- [1] L. Caires and F. Pfenning. Session types as intuitionistic linear propositions. In *CONCUR 2010*, LNCS **6269**, pp. 222–236. Springer, 2010.
- [2] P. Wadler. Propositions as sessions. *Journal of Functional Programming* **24**(2–3):384–418, 2014. Conference version in *ICFP 2012*, pp. 273–286.
- [3] J.-Y. Girard. Linear logic. *Theoretical Computer Science* **50**(1):1–101, 1987.
- [4] R. Milner, J. Parrow, and D. Walker. A calculus of mobile processes, I. *Information and Computation* **100**(1):1–40, 1992.
- [5] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science* **141**(5):49–67, 2005.
- [6] M. Hennessy and R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM* **32**(1):137–161, 1985.
- [7] D. Miller, G. Nadathur, F. Pfenning, and A. Scedrov. Uniform proofs as a foundation for logic programming. *Annals of Pure and Applied Logic* **51**(1–2):125–157, 1991.
- [8] V. A. Saraswat, M. Rinard, and P. Panangaden. Semantic foundations of concurrent constraint programming. In *POPL 1991*, pp. 333–352. ACM, 1991.
- [9] G. Berry and G. Boudol. The chemical abstract machine. *Theoretical Computer Science* **96**(1):217–248, 1992.
- [10] J. Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science* **96**(1):73–155, 1992.
- [11] L. W. Buss. *The Evolution of Individuality*. Princeton University Press, 1987.
- [12] L. G. Meredith. Cost accounting for the rho calculus; the cost endofunctor on continued interactive GSLTs; mortal computation and metabolism. F1R3FLY.io technical notes (this series), 2026.